INTRODUCTION · 引言

> You are holding a book about a tool that did not exist three years ago. Claude Code shipped to general availability in early 2025, and within twelve months it had transformed how a substantial slice of working software engineers spend their days. The transformation is still in progress. Most engineers using Claude Code today are using it the way most people use a new tool: by typing requests at it and accepting whatever it produces. The handful of engineers who have moved past that posture, who have learned to engineer their environment rather than merely use it, are working at a different level of leverage entirely.

你手中这本书所讲的那个工具,三年前还不存在。Claude Code 于 2025 年初正式发布(GA),并在短短十二个月内,改变了相当一部分在职软件工程师的日常工作方式。这一变革至今仍在持续。如今大多数使用 Claude Code 的工程师,用它的方式正和大多数人对新工具的态度一样:敲一段请求,然后接受它吐出的任何结果。而少数已经跨过这道坎的工程师——他们不再只是"用"工具,而是学会"工程化"自己的环境——所发挥出的杠杆效率,根本不在同一个量级。

> This book is for the engineers who want to make that move. The ones who have noticed that their session quality varies wildly day to day. The ones who have wished they could teach Claude their team's

> conventions just once and have those conventions stick. The ones who have watched Claude make the same kind of mistake three times in one afternoon and wondered whether there is a way to make it stop. The ones who suspect that the tool is capable of more than they are getting from it, and who are tired of suspecting and want to start engineering.

这本书写给的就是想迈出这一步的工程师:那些察觉到自己每次会话的质量日复一日忽高忽低的人;那些希望能"教 Claude 一次团队规约,就让它一直守规矩"的人;那些眼见 Claude 一个下午犯三次同类错误、想知道有没有办法让它停下来的人;那些已经隐隐感觉这工具能发挥出远超当下表现的威力,却厌倦了只是怀疑、想要开始动手工程化的人。

> The argument of this book is that Claude Code, properly understood, is not a chat interface. It is a programmable substrate with three distinct customization mechanisms, and the engineers who learn to use those mechanisms in combination produce results that engineers using the chat interface alone cannot match. The three mechanisms have specific names. Skills are units of reusable capability that teach Claude how to do specific kinds of work consistently. Subagents are isolated specialist contexts that handle particular roles in parallel. Hooks are deterministic shell commands that fire at lifecycle moments and impose mechanical guarantees the model cannot evade. Each of these is documented in Anthropic's official materials. None of them, individually, is the secret. The secret, if there is one, is in their composition.

本书的核心论点是:真正理解之后,Claude Code 不是一个聊天界面。它是一个可编程的底层平台(substrate),自带三种截然不同的定制机制;而学会把这三种机制组合使用的工程师,所能产出的成果,是"只用聊天界面"的工程师无论如何也达不到的。这三种机制各有其名。**Skills(技能)**是可复用的能力单元,教 Claude 如何一致地完成某类工作。**Subagents(子代理)**是隔离的专家上下文,并行处理特定角色。**Hooks(钩子)**是在生命周期节点触发的确定性 shell 命令,用来施加模型无法规避的机械性约束。这三者,每一项在 Anthropic 官方文档里都有介绍。但单独任何一项,都不是秘诀所在。如果有秘诀,那秘诀在它们的*组合*。

> What this book teaches, across twenty-two chapters, is the architecture for that composition. Part One lays out the framework. Parts Two through

全书二十二章,讲的就是这套"组合"的架构。第一部分铺陈整体框架;第二至

Four go deep into each pillar. Part Five shows the composed architecture at work in two full case studies, one for a senior engineer at a small product company and one for a solo SaaS founder running an entire business out of a single Claude Code installation. Part Six closes with the practical concerns of shipping, sharing, and evolving an environment over time. The four appendices give you reference material to come back to.

四部分深入每一根支柱;第五部分通过两个完整案例展示组合架构的实战——一个面向小型产品公司的高级工程师,一个用一套 Claude Code 撑起整个生意的独立 SaaS 创始人;第六部分收束于发布、共享、长期演进这些工程化议题;四个附录则是你回头查阅的参考材料。

A short word about who this book is not for. If you are looking for a quick introduction to Claude Code, this is not it. The official Anthropic quickstart is better for that purpose, and it is free. If you are looking for a tour of every feature in the product, this is not that either. The book covers the three customization pillars in depth and treats other features only when they intersect with the three pillars. If what you want is a comprehensive feature catalog, the official documentation is more current than any book can be, and it should be your first stop.

简单说一句"这本书不适合谁"。如果你在找一份 Claude Code 的快速入门,这本书不是——官方的 Anthropic quickstart 更适合这个目的,而且免费。如果你想要一份"产品所有特性"的导览,这本也不是。本书深入讲的是"三根定制支柱",只在与其他特性与这三根支柱相交时才会顺带提及;如果你想要的是"完整的特性目录",官方文档永远比任何书都新,那应该是你的第一站。

What this book offers, that the documentation does not, is a coherent way of thinking about how the pieces fit together. The documentation explains each piece accurately. It does not, and cannot, explain why the three pieces sit at different layers of the architecture, what kinds of failures each pillar prevents, in what order to build them, or how to compose them into a working environment. That synthesis is what this book exists to provide. The pieces are public. The architecture is what you are paying for.

本书提供、而文档没提供的,是一种"这些碎片到底如何拼起来"的连贯思维方式。文档能精确解释每一块;但它无法、也无意去解释:为什么这三块在架构里位于不同层级?每根支柱各防住了哪类失败?应该按什么顺序去搭?又该怎样把它们组合出一个"能跑起来"的环境?本书存在的全部意义,就是提供这种"综合"。那些碎片是公开的;你愿意为这本书付费,买的是这套*架构*。

> A note on how to read the book. The chapters are designed to be read in order, because each pillar builds on the conceptual vocabulary established in the previous one. A reader who jumps directly to the hooks chapters before reading the skills chapters will find the hooks discussion harder than it needs to be, because the deterministic-versus-probabilistic framing that organizes the hooks treatment is set up earlier. That said, after a first read, the book is meant to be returned to non-linearly. The appendices are reference material. The case studies are templates to come back to when you are designing your own environment. Many readers will read the book once cover to cover, then keep it within reach as they build, opening it to specific sections when specific design questions surface.

关于怎么读这本书,多说一句:各章是按顺序写的——因为每根支柱都建立在前面那一根所确立的"概念词汇"之上。如果一位读者跳过 skills 直接去看 hooks,他/她会觉得 hook 那几章比实际更难看懂——因为贯穿 hook 部分的"确定性 vs 概率性"那套框架,是前面就搭好的。当然,在第一遍之后,这本书就是一本"可以非线性回头翻"的书。附录是参考材料;案例研究是"等你设计自己的环境时回来翻的模板"。许多读者会先把整本书通读一遍,然后把它放在伸手够得到的地方;真正开始搭建的时候,遇到具体的设计问题再翻开对应的小节。

> A note about pronouns. The hypothetical developer throughout the book is referred to with feminine pronouns. This is a deliberate choice and is meant to be read as one. Software engineering is more diverse than its

关于代词的说明:全书那位假想中的"开发者",统一使用*她*。这是一项经过思考的、有意为之的选择,读到这里请按其本意来读。软件工程比它通常被描绘的代词所指代的群体,要更多元。

> conventional pronouns suggest, and a book written in 2026 ought to reflect that reality. Readers who find this jarring may want to take a moment to consider why. The book itself proceeds without further comment.

一本写于 2026 年的书,理应反映这一现实。如果有读者觉得这种写法有些刺眼,不妨花一点时间想想"为什么";本书对此不再赘述。

> A note about the present moment. This book is being written and shipped at a particular point in the evolution of AI-assisted development tooling. The specific features the book describes are accurate as of early 2026. Some of those features will change. New features will appear. The framework, however, is meant to be durable, because it is rooted in the structure of engineering work itself rather than in the specifics of any particular tool. A reader returning to this book in 2028 will find that some of the syntactic details have moved on, but that the architectural principles still apply, because the underlying division of capability, specialization, and enforcement is a property of the work and not of the tool.

关于"此刻"的几句话:本书写作与面世,正逢 AI 辅助开发工具演化的某个特定节点。本书描述的那些具体特性,在 2026 年初是准确的;其中一些会变,新特性也会出现。然而,这套框架的*意图*是耐久的——因为它根植于"工程工作本身的结构",而不是任何特定工具的细节。等 2028 年回头再翻这本书的读者,会看到:一些语法细节已经过时了;但架构层面的原则仍然成立——因为"能力的划分、专门化的划分、强制的划分"那套底层结构,是"工作"本身的属性,不是"工具"的属性。

> Now to the work. The first chapter opens at the moment most engineers experience as the trigger for taking their tooling more seriously: a Friday afternoon, three different kinds of failure in quick succession, and the dawning recognition that the way they have been using the tool is not going to scale to the work they actually need to do.

现在开工。第一章,会从大多数工程师"开始认真对待自己工具"那一刻切入:一个周五的下午,接连发生的、互不相同的几类失败,以及"我一直在用的方式,撑不起我真正要做的工作"这一正在浮出水面的认知。

PART ONE · 第一部分

Foundations: The Three Pillars of Claude Code Architecture

基础:Claude Code 架构的三根支柱

The gap between using Claude Code and engineering with Claude Code.

"使用 Claude Code"与"用 Claude Code 做工程"之间的距离。

CHAPTER ONE · 第一章

Why Claude Code Alone Isn't Enough

为什么单靠 Claude Code 远远不够

> *"Tools are not merely aids to the hand. They are extensions of the mind, and the mind that fails to shape them is shaped by them instead."*
>
> — Marshall McLuhan, paraphrased

> *"工具绝不仅仅是手的延伸,更是心智的延伸;不去塑造工具的心智,反过来必被工具所塑造。"*
>
> —— 马歇尔·麦克卢汉(转述)

There is a specific moment, familiar to anyone who has lived with Claude Code for more than a few weeks, when the honeymoon ends. The tool still feels magical. The responses are still sharp. But the reader begins to notice something quieter underneath the surface: the same instructions are being typed again. The same three warnings are being issued before every refactor. The same linting step is being requested after every edit. The same four sentences of project context are being pasted into the first message of every new session. The magic is still there, but it has started to feel like a magic that requires a great deal of bookkeeping.

凡是跟 Claude Code 相处过几周以上的人,都会熟悉一个特定时刻——蜜月期结束的那一刻。工具依然令人惊艳,响应依然敏锐。但在表面之下,读者开始察觉到一些更安静的迹象:同一段指令在反复输入;同一套三次警告在每次重构前都重新出现;同一步 lint 在每次编辑后都被重新请求;同一段四句长的项目背景在每个新会话开头被原封不动地粘贴。魔力依然在,但它已经开始变成一种需要大量"簿记工作"才能维持的魔力。

This book was written for that moment. Not the moment of discovery, when Claude Code first opens in a terminal and writes a working function from a vague sentence. Not the moment of curiosity, when the

> reader begins to explore what the tool can and cannot do. The moment this book was written for is the moment of friction. The moment when the gap between using the tool and being mastered by the tool becomes visible, and when the reader begins to suspect, correctly, that there must be a better way.

这本书就为那个时刻而写。不是"发现"那一刻——Claude Code 第一次在终端里被打开、从一句模糊的话写出能跑的函数;也不是"好奇"那一刻——读者开始试探这工具能做什么、不能做什么。本书写的是*摩擦*那一刻:当你看清了"用工具"和"被工具所用"之间的鸿沟,并开始正确地怀疑——一定还有更好的方式。

> The better way is not a better prompt. It is not a longer `CLAUDE.md` file. It is not a collection of tricks. The better way is a shift in how the reader relates to the tool itself: from a user of Claude Code to an engineer of Claude Code. This shift is the central promise of the book, and this chapter is where that promise gets defined.

那条更优的路,不是写一段更好的 prompt,也不是把 `CLAUDE.md` 写得更长,更不是堆一摞小技巧。所谓更优,是关系本身的转变:从 Claude Code 的*使用者*,变成 Claude Code 的*工程师*。这一转变,是本书最核心的承诺;而这一章,正是把那个承诺明确下来的地方。

> Consider a concrete scene. A developer sits down on a Tuesday morning to continue work on a medium-sized web application. She opens Claude Code. The first thing she types, as she has typed roughly forty times before, is a reminder that this codebase uses TypeScript with strict mode, that components should be written as functional components with hooks, that tests are written in Vitest rather than Jest, that commit messages follow the conventional commits format, and that any changes to the authentication layer require her personal review. She types this reminder because she has learned, through painful experience, that if she does not type it, Claude Code will occasionally produce perfectly reasonable code that violates every one of those rules.

请设想一个具体场景。一位开发者在某个周二上午坐下,继续开发一个中等规模的 Web 应用。她打开 Claude Code,敲下的第一段话——和之前大约四十次一样——是重新提醒一遍:这个代码库使用 TypeScript 的 strict 模式;组件必须写成带 hooks 的函数式组件;测试用 Vitest,不要用 Jest;commit message 遵守 conventional commits 规范;鉴权层的任何修改必须经她本人复核。她之所以要打这一遍,是因为她吃过大亏才学到的:不打这一段,Claude Code 偶尔会写出一段"看起来很合理、却把所有规则都违反一遍"的代码。

> She then asks Claude Code to add a new feature. The code appears. It is, as always, impressive. She reads it. She notices that the test file is missing, because she forgot to ask for it. She asks for tests. The tests appear.

接着她让 Claude Code 加一个新功能。代码出来了,一如既往地漂亮。她通读一遍,发现测试文件缺了——因为她刚才忘了开口要。她补上"给我测试",测试出来了。

> She notices that one test uses a matcher that exists in Jest but not Vitest. She corrects Claude, who apologizes and fixes it. She commits the change. She notices, twenty minutes later, that her commit message does not follow the conventional commits format, because she wrote it herself in a hurry. She amends the commit.

她发现,其中一个测试用到的 matcher 只在 Jest 里有,Vitest 没有。她纠正 Claude,Claude 道了歉、改掉了。她提交了改动。二十分钟后,她注意到自己的 commit message 并不符合 conventional commits 规范——那是她刚才匆忙之下自己写的。她又补了一次 commit。

The Working Developer's Shape · 普通开发者的早晨

> This is a good developer. She is getting real work done. Claude Code is, objectively, saving her hours. And yet, look closely at the shape of her morning. It is full of tiny, avoidable frictions. Instructions she typed that should have been remembered. Checks she ran that should have fired automatically. Specialists she consulted in her own head, one after another, because her tool did not know how to consult them for her. She is doing the work that a well-engineered environment should be doing on her behalf.

这是一位很靠谱的开发者。她确实在产出真实的工作。客观地讲,Claude Code 替她省下了大量时间。然而,若细细打量她上午的工作流,你会看到:处处都是细碎的、本可避免的摩擦——本该被记住的指令,本该自动触发的检查,本该由"专家"来回答的问题——这些她都得在自己脑子里一个个处理,因为她的工具不知道如何替她去请那些"专家"。她正在替那个本应被工程化好的环境,做着它本该做的事。

> Now imagine the same morning in a different configuration of the same tool. She opens Claude Code. She does not type a reminder about TypeScript or Vitest or commit messages, because the environment already knows. When she asks for a new feature, the code appears with tests, because a skill for writing features in this codebase has been installed and knows that tests are part of the definition of done. Before anything is written to disk, a pre-edit hook silently checks the file path against the codebase's protected paths. After the code is written, a post-edit hook runs the linter and the type checker, and if they fail, Claude Code sees the failure and corrects the code before returning control to her. When she is ready to commit, a pre-commit hook runs the full test suite in the background, and a subagent dedicated to commit messages produces a conventional commit message from the diff. She reviews the message, accepts it, and moves on.

现在,请想象同一个上午、同一套工具,但环境配置完全不同。她打开 Claude Code,却不再需要重复提醒 TypeScript、Vitest、commit message 这些——因为环境已经知道了。她提一个新功能,代码连同测试一起出现,因为已经安装了一个"为本代码库写功能"的 skill,它知道"完成"里包含测试。文件落到磁盘之前,有一个 pre-edit hook 默默核对路径,确认没碰受保护目录。代码写完后,有一个 post-edit hook 跑 linter 和类型检查器;若失败,Claude Code 自己看到失败,先改好,再把控制权交还给她。她准备提交时,一个 pre-commit hook 在后台跑完整套测试;一个专门负责 commit message 的 subagent 根据 diff 写出一段 conventional commit 信息。她浏览一遍,通过,然后继续。

> The work done in this second configuration is the same work. The tool is the same tool. What has changed is the scaffolding around it. The developer has stopped being the carrier of every rule and every check, and has become, instead, the supervisor of a small, patient, consistent system that carries those rules and checks on her behalf.

两种配置下,做的事情是同一件事,工具是同一个工具。变的,是工具周围的那层脚手架。开发者不再背负每一条规则、每一道检查;她变成了一个小型、耐心、始终如一的系统的*监督者*,由那个系统替她背着这些规则和检查。

> The distance between those two mornings is the distance between using Claude Code and engineering with it. And that distance is not a matter of talent, intelligence, or time spent with the tool. It is a matter of knowledge. The second developer knows something the first one does not. She knows that Claude Code is not a chat interface to a language model. She knows that Claude Code is a programmable environment, and she knows the three systems that make the programming possible.

这两个上午之间的距离,就是"使用 Claude Code"和"用 Claude Code 做工程"之间的距离。而这段距离,与天赋、智商、与工具相处的时间都无关。它是*认知*的差距。第二位开发者知道一些第一位不知道的事:Claude Code 不是通往语言模型的聊天界面;Claude Code 是一个*可编程的环境*;并且,她了解让这种编程得以成立的那三套系统。

> that Claude Code is not a chat interface to a language model. She knows that Claude Code is a programmable environment, and she knows the three systems that make the programming possible.

知道 Claude Code 不是一个通往语言模型的聊天界面;知道 Claude Code 是一个*可编程的环境*;并且,她了解让这种编程得以成立的那三套系统。

> Those three systems are Skills, Subagents, and Hooks. Every power-user setup, every company-wide Claude Code deployment, every open-source configuration pack that developers are quietly building and trading in private Slacks right now, is built out of some combination of these three primitives. They are the primitives of Claude Code mastery. They are also, surprisingly, the primitives that the overwhelming majority of Claude Code users have never consciously touched.

这三套系统,就是 Skills、Subagents、Hooks。每一个高阶用户的配置、每一次企业级的 Claude Code 部署、每一份开发者们正在悄悄搭起来、并在私有 Slack 群里互通有无的开源配置包,都是由这三种原语的某种组合搭建出来的。它们是掌握 Claude Code 的"原语"。而令人惊讶的是:绝大多数 Claude Code 的用户,从未有意识地触碰过它们中的任何一个。

> This is the strange fact at the center of the tool. Anthropic shipped Claude Code with all three of these systems built in from an early stage. The documentation exists. The configuration files are well-designed. The examples in the official repositories are good. And yet, walking into any developer forum or any Claude Code community channel, the overwhelming pattern of conversation is about prompt wording. About trying different `CLAUDE.md` phrasings. About sharing clever one-liners. The three systems that could transform a workflow go almost entirely unmentioned, because they sit one level below the surface, and almost no one has taken the time to descend.

这就是这个工具中央那个反常的事实。Anthropic 在 Claude Code 早期就把这三套系统全部内置了。文档在;配置文件设计得不错;官方仓库里的样例也写得到位。然而,随便走进任何一个开发者论坛或任何一个 Claude Code 社区频道,话题压倒性地围着 prompt 措辞打转——尝试不同的 `CLAUDE.md` 写法,分享巧妙的一行 prompt。其实可以彻底改变工作流的那三套系统,几乎完全无人提及,因为它们藏在表面之下整整一层,几乎没人愿意花时间潜下去看一眼。

Three Systems, One Architecture · 三个系统,一套架构

> The reader is about to descend. But first, it is worth naming the three systems clearly, so that every subsequent chapter has a stable vocabulary to refer back to.

读者即将潜下去。但在进入正文之前,有必要把这三套系统的名字讲清楚——这样后续每一章都会有一个稳定可援引的词汇表。

> A Skill is a reusable capability. It is a folder containing a `SKILL.md` file, and possibly supporting scripts or reference material, that teaches Claude how to perform a specific task consistently. When a user asks Claude to perform that task, Claude recognizes the request, reads the skill, and carries it out according to the skill's definition. Skills are how a developer stops repeating themselves. They are the mechanism by which institutional knowledge enters the environment and stays there.

Skill(技能)是一种可复用的能力。它是一个目录,里面有一个 `SKILL.md` 文件,可能还附带支持脚本或参考材料,用来教 Claude 如何一致地完成一项特定任务。当用户请 Claude 做这件事时,Claude 识别出请求,读取该 skill,按其定义执行。Skill,是开发者停止重复自己的方式;也是组织知识进入这个环境、并长期留存的机制。

> A Subagent is a specialized parallel worker. It is a separate Claude process, with its own system prompt, its own restricted set of tools, and its own isolated context window. The main Claude session delegates work to a

Subagent(子代理)是一个专门化、可并行的工作者。它是一个独立的 Claude 进程,有自己的系统提示、受限的工具集,以及隔离的上下文窗口。主会话把工作委派给一个

> subagent, the subagent performs the work, and only the final result comes back. Subagents are how a single developer becomes the orchestrator of a small team of specialists without paying the cognitive cost of switching roles. A code-reviewer subagent reviews code. A test-writer subagent writes tests. A security-auditor subagent audits security. They do not interrupt each other, they do not pollute each other's thinking, and they do not consume the main session's context window.

subagent,subagent 执行,只把最终结果回传。Subagent,是一个人如何"在不付认知切换成本的前提下"变成一支小型专家团队编排者的方式。一个 code-reviewer subagent 负责审查代码;一个 test-writer subagent 负责写测试;一个 security-auditor subagent 负责安全审计。它们互不打断、互不污染思考、也不消耗主会话的上下文窗口。

> A Hook is a deterministic event trigger. It is a shell command, bound to a specific moment in the Claude Code lifecycle, that executes automatically whenever that moment occurs. A hook fires when a tool is about to be used, or just after it was used, or when a session starts, or when it ends. Hooks are how a probabilistic system gets wrapped in deterministic edges. They are how the developer stops trusting Claude to remember to run the tests and starts trusting the hook that always runs the tests whether Claude remembers or not.

Hook(钩子)是一个确定性的事件触发器。它是一条 shell 命令,绑定到 Claude Code 生命周期中的某个具体时刻,只要该时刻发生,它就自动执行。工具被调用之前、之后、会话开始、会话结束——hook 都可以触发。Hook,是"如何用一个确定性的外壳把一个概率性的系统包起来"的方式。它把"信任 Claude 自己记得跑测试"换成"信任那个无论 Claude 记不记得都会跑测试的 hook"。

> Each of these systems, on its own, is useful. A skill saves the developer from typing the same instructions twice. A subagent saves the context window from drowning in noise. A hook saves the workflow from the failure mode of forgetting. But the real power, the power that this book is built around, does not come from the three systems in isolation. It comes from the composition. It comes from building an environment in which all three work together, reinforcing each other, filling each other's gaps, turning Claude Code into something that is less like a tool the developer uses and more like a development environment that the developer has engineered to fit the shape of her work.

这三套系统,每一套单拿出来都很有用:Skill 让你不必打两遍同样的指令;Subagent 让主会话的上下文窗口不被噪声淹没;Hook 让工作流从"忘记"这种失败模式中解放出来。但真正的力量——也是

本书所围绕的核心——并非来自任何一套独立使用,而是来自*组合*。来自"让三者协同工作、互相补位、互相加固",把 Claude Code 从"一个开发者使用的工具"变成"一个开发者按自己工作形状工程化出来的开发环境"。

> That phrase, fully engineered, is not an exaggeration. By the time the reader has finished this book, the reader will have installed skills that teach Claude how to write code the way the reader wants it written. The reader will have defined subagents that handle specialized work without disturbing the main session. The reader will have configured hooks that enforce the boundaries and the rituals of the reader's workflow without any conscious effort. The three systems will be humming together, quietly, behind every prompt the reader types.

所谓"被完整工程化",绝非夸张。当你读完这本书时,你将:装上若干 skill,教 Claude 按你想要的风格写代码;定义若干 subagent,让它们处理专项工作而不打扰主会话;配置好若干 hook,让工作流的边界和仪式感无须刻意维护、就自动生效。三套系统,会在你每敲下一句 prompt 的背后,安静地一起运转。

Engineering as Platform · 把工程做成"平台"

> There is a word for this in software engineering: platform. A platform is the set of conventions, tools, and automations that shape how work is done within an organization, so that individual engineers do not have to carry all of that shaping in their own heads. What this book teaches, in the end, is how to build a personal platform on top of Claude Code. Not a company platform, necessarily, though the same techniques scale. A personal platform. One developer, one machine, one Claude Code installation, and a set of skills, subagents, and hooks that together make the environment feel as if it was built specifically for the way that one developer thinks.

软件工程里有一个词来形容这件事:平台(platform)。所谓平台,就是一套约定、工具、自动化机制,它们共同塑造了一个组织里"工作是怎样完成的",从而让单个工程师不必把这些塑造规则都装在自己脑子里。本书最后要教的就是:如何在 Claude Code 之上搭一个"个人平台"。不一定非要是公司级平台——虽然这套技术同样可以放大到公司级。是*个人平台*。一位开发者,一台机器,一套

Claude Code 安装,再加上一组 skills、subagents、hooks——让整个环境看起来仿佛就是为这一位开发者的思维方式量身打造的。

> This is not a small thing to promise. And it is worth being honest about what it costs. Building a personal Claude Code platform requires an investment. Not a large one, but a real one. The developer has to understand the tool well enough to see which frictions are worth automating and which are not. The developer has to be willing to write configuration as carefully as she writes production code, because the configuration is, in effect, production code for her own workflow. The developer has to treat her `.claude` directory with the same respect she treats her source code, versioning it, reviewing changes, and improving it over time.

这并不是一个轻飘飘的承诺。所以有必要坦白它的代价。搭一个个人 Claude Code 平台,需要投入——不是巨额,但必须真实。开发者需要足够了解这个工具,才能看清哪些摩擦值得自动化、哪些不值得。她需要愿意像写生产代码那样认真地写配置——因为那份配置,在事实上,就是她自己工作流的生产代码。她需要像对待源代码一样对待自己的 `.claude` 目录:版本化、审阅变更、持续迭代。

> In exchange for that investment, the developer gets something that compounds. Every skill installed pays dividends for every future session in which that skill fires. Every subagent defined saves the context window on every future task that would otherwise have polluted it. Every hook installed protects every future action from the failure mode it was designed to prevent. A year of thoughtful configuration turns into an environment that is noticeably more productive than any colleague's environment who has not made the same investment. Two years of it turns into a compounding lead that becomes difficult to close.

作为对这份投入的回报,开发者会得到一种*复利*。每装一个 skill,都会在它今后触发的每一个 session 里产生收益。每定义一个 subagent,都会在今后每一个本会污染上下文的任务中省下主会话的窗口。每配置一个 hook,都会在今后每一个动作中预先挡掉它专门针对的那种失败。一年的深思熟虑的配置,会让这个环境比"没做同样投入"的同事的环境显著高产;两年下来,这种复利就会变成一道难以追平的领先。

> This is the pitch of the book. It is not a pitch about tricks. It is a pitch about building a platform. The reader who follows the path laid out in these chapters will finish with a platform of her own, and with

the understanding of how to extend that platform indefinitely as her work changes and as Claude Code itself evolves.

这就是本书的卖点。它不是在兜售"小技巧",而是在兜售"搭一个平台"。沿着这些章节铺好的路径走下来的读者,最终会拥有自己的平台,以及一种能力:随着工作变化、随着 Claude Code 自身演化,持续、无限制地扩展那个平台。

The rest of Part One is about laying the foundations for that journey. Chapter Two takes the reader under the hood of Claude Code, because no serious engineering is possible without an accurate mental model of the

第一部分剩下的内容,都在为这段旅程打地基。第二章会带你掀开 Claude Code 的引擎盖,看它内部是怎么运转的——因为没有准确的心智模型,严肃的工程化无从谈起。

system being engineered. Chapter Three introduces the three-pillar framework formally, explains how the pillars map to the real problems of daily development, and establishes the composition principle that will guide every subsequent part of the book. By the end of Part One, the reader will have the vocabulary and the mental model needed to understand why the rest of the book says what it says.

正在被工程化的那个"系统",到底是什么样的。第二章之后,第三章会正式介绍"三支柱"框架,讲清这三根支柱是怎么映射回日常开发的真实问题,并奠定将贯穿全书后半段的"组合原则"。读完第一部分,你将拥有阅读后续内容所必需的词汇和心智模型,理解为什么后面的章节要这么写、这么说。

It is worth pausing here on a question the reader may already be asking. If the three systems are so valuable, why are they underused? The question has a few honest answers, and each of them is worth naming, because naming them helps the reader avoid the fate of the developers who have not benefited from what the tool already offers.

这里值得停下来,回答一个读者可能已经在心里冒出来的问题:既然三套系统这么有价值,为什么用的人这么少?这个问题有几个诚实的答案,每一个都值得点出——因为点出它们,能帮你避开"本可以

受惠于工具却没受惠"的开发者的命运。

Why So Few Have Done It · 为什么做的人这么少

> The first answer is visibility. Prompts are visible. A developer sees her own prompts every day, and she sees the prompts that other developers share in public channels, and she forms an intuition that prompting is where the action is. Configuration is invisible. A developer does not see another developer's `.claude` directory by default. It does not show up in screencasts. It does not appear in most tutorials. It is the dark matter of Claude Code: the thing that holds the visible workflow together, but that the visible workflow never calls attention to. The natural result is that developers imitate what they can see, and what they can see is mostly prompts.

第一个答案是*可见性*。Prompt 是可见的:开发者每天都能看到自己写的 prompt,也能在公共频道看到别人分享的 prompt,从而形成一个直觉——"功夫在 prompt"。配置是不可见的:开发者默认看不到别人的 `.claude` 目录;它不会出现在录屏里;它也不会出现在大多数教程里。它就是 Claude Code 的"暗物质":那根把可见的工作流撑起来的梁,但可见的工作流从不主动提到它。自然的结果就是——开发者模仿她能看见的东西,而她能看见的,主要是 prompt。

> The second answer is that configuration feels like overhead. A developer who is trying to ship a feature does not want to stop shipping features in order to design a skill library. The investment looks like a distraction from the work, and the returns are deferred. This feeling is real, and it is also wrong. The returns are not deferred by very long. A well-designed skill pays for itself within a week of normal use, because it runs automatically for every matching task and saves its entire cost every time. A hook pays for itself the first time it prevents a failure that would have taken an hour to clean up. The problem is not that the returns are slow; it is that the returns are distributed across many future sessions, rather than concentrated in the session where the investment was made, so they are psychologically harder to see.

第二个答案是*配置看上去像额外负担*。一个正忙着交付功能的开发者,不想为了设计一套 skill 库而停下来。投入看起来像是工作的"分心",回报又像是延迟到未来的。这种感觉是真实的,但也是错的。回报并不会真的"延迟很久":一个设计良好的 skill,正常使用一周内就能收回成本,因为它会为每一个匹配的任务自动运行,每次都把全部成本省下来;一个 hook,在它第一次阻止了一次本来要花一小时收拾的故障时,就已经回本。问题不在于回报慢;问题在于回报分散在许多未来的 session 里,而非集中在"投入发生"的那个 session 里——所以从心理上更难被看见。

> The third answer is cultural. The habit of configuring tools carefully used to be widespread in software engineering, and it has eroded.

第三个答案是*文化*。"认真配置工具"这个习惯,曾经在软件工程里普遍存在,而今已经逐渐消退。

> Developers customize their editors, but they do not usually customize their terminals, their shells, or their build systems with the same care that was once standard. Claude Code is a tool that rewards that older discipline. It is designed to be bent and shaped, and it gives back, in long-term productivity, whatever the developer is willing to put in during the shaping. Readers who bring the older discipline back with them into this new tool will find themselves at a significant advantage over those who do not.

开发者会定制自己的编辑器,但已经不太用同样的心力去定制终端、shell、构建系统——而这在过去是标配。Claude Code 是一个会回报那种"老派严谨"习惯的工具。它生来就允许被弯折、被塑造;你愿意在塑造上投入多少,它就在长期生产力上以相应的形式回馈你多少。把那份老派严谨重新带回到这个新工具里的读者,会显著领先于没带回来的人。

> One more anchor is worth placing before this chapter ends. Consider what Claude Code would feel like if all three pillars were ignored. A developer in that situation is using Claude Code as a conversational interface to a language model. She gets good outputs when she writes good prompts. She gets bad outputs when she writes bad prompts. Her productivity rises and falls with her attention, her memory, and her patience. Now consider the same developer after six months of thoughtful configuration. Her productivity no longer rises and falls in the same way, because the environment has absorbed much of the burden that used to sit on her attention. The ceiling of what she can produce in a day has risen, because the floor has also risen. On her worst days, the environment carries her. On her best days, the environment amplifies her. The variance has fallen and the average has climbed. This is, in the end, the simplest case for engineering rather than merely using: a reduction in variance and an increase in average, across every future session the developer will ever run.

在本章结束之前,值得再立一个锚点。请设想:三根支柱全部被忽略时,Claude Code 用起来是什么感觉。那位开发者把它当成了一个通往语言模型的对话界面:prompt 写得好,产出就好;prompt 写得差,产出就差;她的生产力,完全随着她的注意力、记忆力和耐心上下波动。再设想:同一位开发者,

在经过六个月的认真配置之后,她的生产力已经不再以同样的方式起伏了——因为环境把她原本要背负的许多东西都吸走了。她一天能产出的"上限"被抬高了,因为"下限"也被抬高了。状态最差的日子里,环境托着她;状态最好的日子里,环境放大了她。方差下降了,均值上升了。说到底,"工程化"而不是"只是用",最简洁的论据就是这一点:在她未来要跑的每一个 session 里,方差降低、均值提高。

> There is a deeper argument to be made, too, about what this kind of configuration does to a developer over time. A developer who has spent six months shaping her Claude Code environment has, in the course of doing so, answered a series of questions about her own work that she might otherwise never have answered explicitly. What is my preferred format for a pull request description? What counts as a complete definition of done for a new feature in my codebase? Which files should never be edited without manual confirmation, and why? What does a good commit message look like in this project, precisely? Every skill, subagent, and hook the developer builds is an answer to one of these questions, captured in a form the tool can use. The side effect of building the environment is that the developer ends up with a much clearer articulation of how she actually wants to work than she had before she started. The environment is a mirror of the developer's craft, and the craft becomes sharper by being mirrored. This is a

还有一个更深的论点:这种配置,长期看,会对一个开发者本身做些什么。花六个月去塑造自己 Claude Code 环境的开发者,在塑造的过程中,实际上回答了一连串关于自己工作的、她以前可能永远不会明确作答的问题——"我偏好的 PR 描述格式是什么?""在我的代码库里,一个新功能的'完成'到底包括哪些?哪些文件绝对不能不经我手就改,为什么?""在这个项目里,一段好的 commit message 究竟长什么样?"每一个她搭建起来的 skill、subagent、hook,都是这些问题的某一条答案,只是以工具能使用的形式固化下来。搭建环境的副作用是:开发者最终对自己"到底想怎么工作"有了一份比开始之前清晰得多的表达。环境是开发者手艺的一面镜子,而手艺因被映照而变得更锐利。这是一种

> benefit that never appears on any productivity chart, but it is one of the most valuable outcomes of the whole exercise, and readers who go through the work will recognize it when it arrives.

它从来不会出现在任何"生产力图表"上,却是整件事最有价值的成果之一——走过这段路的读者,会在它真正到来的那一刻认出它。

> The remaining frustration of the Tuesday morning described at the start of this chapter is not an inherent property of working with Claude Code. It is a property of working with Claude Code without having engineered the environment around it. That environment, the one the second developer was enjoying, is buildable by anyone. The building is what this book is about. The next step in the building is understanding the machine we are about to shape, and that is the subject of the chapter that follows.

本章开头描述的那个"周二上午的沮丧",并不是 Claude Code 本身带来的固有属性——它是"用 Claude Code、却没在它周围搭建起工程化环境"才带来的属性。那个环境,也就是第二位开发者正在享受的那个,任何人都能搭建。搭建的过程,就是本书的全部内容。搭建的下一步,是理解我们即将去塑造的那台机器——而那正是下一章的主题。

← Chapter 1 上一页

Chapter 2: The Anatomy of Claude Code →

CHAPTER TWO · 第二章
The Anatomy of Claude Code · Claude Code 的解剖

> *"If I had an hour to solve a problem, I would spend the first fifty-five minutes understanding the problem, and the last five solving it."*
>
> — attributed to Albert Einstein

> *"如果给我一个小时解决一道难题,我会花前五十五分钟弄清问题本身,最后五分钟再去解题。"*
>
> —— 通常归于阿尔伯特·爱因斯坦

> To engineer something, one must first see it clearly. This chapter is about seeing Claude Code clearly. Not as a conversational interface, which is how it feels from the outside, but as a layered system with a specific shape, a specific set of moving parts, and a specific vocabulary for the relationships between those parts. Without this clarity, every later chapter in the book will feel like a collection of incantations. With it, every later chapter will feel like the deliberate, ordinary application of a few simple principles to a machine whose behavior is already understood.

要工程化一个东西,首先得先把它看清楚。这一章,就是要把 Claude Code 看清——不是从外面看过去那种"像聊天界面"的观感,而是把它当成一个分层的系统:有它特定的形状、特定的运动部件、特定的词汇来描述这些部件之间的关系。少了这份"看清",后面每一章都会像一堆咒语的合集;有了它,后面每一章都会变成:对一台"我们其实已经懂得其行为"的机器,几条简单原则的从容而普通的应用。

> The first thing to understand about Claude Code is that, from the moment a session begins, a small but crucial amount of context is being assembled on the user's behalf, every single turn, before the user ever sees a response. The user types a prompt. That prompt is not sent in isolation to the language model. It is sent as part of a carefully constructed bundle. The bundle includes a system prompt that defines who Claude is and what it is authorized to do within this environment. It includes a description of the tools that Claude is allowed to call, including the file system tools, the shell, the web, and whatever other tools the installation has made available. It includes contextual information about the project, typically harvested from files like `CLAUDE.md` or from explicit references the user has made. It includes, when relevant, the definitions of the skills and the subagents that apply to this session, and it includes the configuration that tells Claude which hooks will fire at which points. Only after that entire bundle is constructed is the user's prompt appended, and the whole assembly is sent to the language model for a response.

关于 Claude Code,第一件要理解的事是:从一次会话开始的那一刻起,在用户看到任何一次响应之前,每一轮都会在用户那边"代为装配"出一小撮、但又至关重要的上下文。用户敲下一句 prompt,那句 prompt 并不是"孤身"被送到语言模型那里去的;它是作为一个被精心构造好的"包袱"的一部分被送过去的。这个包袱里包括:一段系统提示(system prompt),定义 Claude 是谁、在本环境里被授权做什么;一份工具清单,告诉模型它可以调用哪些工具——包括文件系统、shell、web,以及安装时给它配置进来的其他工具;关于本项目的上下文信息,通常取自 `CLAUDE.md` 这类文件,或者用户显式给出的引用;在相关的情况下,还会包含本 session 适用的若干 skill 与 subagent 的定义,以及那份"告诉 Claude 在哪些时刻会触发哪些 hook"的配置。直到这一整包都装配完毕,用户的 prompt 才会被追加到末尾,然后把整包一起送到语言模型那里去,等一次响应。

This matters because it means that the user's prompt is not the sole cause of Claude's behavior. The user's prompt is one ingredient among many, and in many cases, it is not even the most important ingredient. The system prompt is more important. The `CLAUDE.md` content is often more

这件事之所以重要,是因为它意味着:用户的 prompt,并不是 Claude 行为的唯一成因。Prompt 只是众多配料中的一种,很多时候,甚至不是最重要的一种。系统提示更重要。`CLAUDE.md` 的内容,通常更

important. The skills that are recognized as applicable are often more important. A developer who believes that the only way to change Claude's behavior is to change the wording of her prompts is a developer operating with a faulty mental model of where behavior comes from. The model is mostly not in the prompt. The model is in the bundle.

重要。被识别为"当前适用"的那些 skill,通常还更重要。一个以为"想改 Claude 的行为只能改 prompt 措辞"的开发者,她对"行为到底来自哪里"的心智模型,从根本上就错了。模型不在 prompt 里——模型在那整包"装配好的上下文"里。

The second thing to understand is that Claude Code is agentic. The word *agentic* has been overused in the last two years to the point of fatigue, but in this specific context it has a precise and useful meaning. An agent, in the sense that matters here, is a language model that has been given the ability to call tools, observe the results of those tool calls, and decide, based on those observations, what to do next. Claude Code is an agent in exactly this sense. When a user asks Claude Code to add a new feature to a codebase, Claude does not produce the code in a single shot and then stop. Claude produces a plan, inspects files to verify assumptions about the codebase, reads surrounding code to understand conventions, writes a first draft of the new code, sometimes runs tests or type checks, observes any failures, and adjusts the code in response. This loop of propose, act, observe, adjust is the beating heart of the tool, and every subsequent system in this book is ultimately a way of shaping what happens inside that loop.

第二件要理解的事是:Claude Code 是*智能体式*(agentic)的。*agentic* 这词在最近两年被用得太滥,滥到让人疲劳;但是在本书的具体语境里,它有一个精确且有用的含义。这里的"agent"(智能体),是

指一个语言模型——它被赋予了一种能力:调用工具、观察这些工具调用的结果、并基于这些观察决定下一步做什么。Claude Code 正是这种意义上的 agent。当用户让 Claude Code 给代码库加一个新功能,Claude 不会"一把梭哈"地把代码写完就停;它会先列计划,检查文件以核实对代码库的假设,读周围的代码以理解约定,写新代码的第一稿,有时还会跑测试或类型检查,观察有没有失败,再据此调整。这条"提议—行动—观察—调整"的循环,就是这款工具的"心脏";本书后面讲的所有系统,归根结底都是"如何塑造这条循环内部所发生的事"。

The Loop · 那个循环

> The loop is worth naming because it will be referred to constantly. The loop has three phases per iteration. In the first phase, Claude thinks, which is to say, the language model produces text that constitutes its next reasoning step. In the second phase, Claude acts, which is to say, Claude chooses a tool to call and provides the arguments to that tool. In the third phase, Claude observes, which is to say, the result of the tool call is read back into the context window, and Claude decides whether it is done or whether another iteration is needed. Think, act, observe. Think, act, observe. The loop runs until Claude decides, based on the current state of the context window, that the task is complete.

值得给这个循环起个名字,因为后面会反复提到它。每迭代一次,这个循环包含三个阶段。第一阶段:Claude*思考*——模型生成一段文本,代表它下一步的推理动作。第二阶段:Claude*行动*——它选一个要调用的工具,并给出这个工具的参数。第三阶段:Claude*观察*——工具调用的结果被读回上下文窗口,Claude 据此判断:这件事是否已经做完了,还是需要再迭代一次。思考,行动,观察;思考,行动,观察……这个循环一直跑,直到 Claude 在当前上下文窗口的状态下判断:任务已经完成。

> This loop is the reason subagents exist, the reason hooks exist, and the reason skills matter. Subagents are a way of saying, this part of the loop is a different kind of work, and it deserves its own isolated loop with different rules. Hooks are a way of saying, whenever the act phase is about to call this specific tool, run this shell command first, or after, to enforce

正是这个循环,回答了"为什么要有 subagent、为什么要有 hook、为什么 skill 重要"。Subagent,是用来表达"循环中的这一段是另一类工作,值得给它一条独立的、规则不同的循环";Hook,是用来表达"无论何时,只要行动阶段即将调用这个具体工具,先(或后)跑这条 shell 命令,以施加某种

something deterministic that the model alone cannot be trusted to enforce. Skills are a way of saying, this kind of task recurs often enough that it deserves a standing set of instructions, so that when the loop encounters it, the right behavior is already present in the context bundle.

确定性——而这种确定性,单靠模型本身是没法被信赖它会执行的"。Skill,是用来表达"这类任务重复出现的频率,已经足以值得为它常驻一套指令;这样当循环遇到它时,正确的行为就预先存在于那包上下文里"。

The third thing to understand is the file system. Claude Code is not a disembodied agent floating above a project. It runs inside a specific working directory, and it is deeply aware of the structure of that directory. A set of conventions controls what Claude sees and how Claude configures itself. The most important of these conventions is the `.claude` directory. The `.claude` directory sits at the root of a project, or in the user's home folder, or in both, and it is where configuration lives. Skills live in `.claude/skills/` as subfolders. Subagent definitions live in `.claude/agents/` as files. Hook definitions live in the settings file at `.claude/settings.json`, or in the corresponding user-level settings file. The structure of this directory is, in effect, the source code of the developer's Claude Code environment.

第三件要理解的事,是*文件系统*。Claude Code 不是一个"漂浮"在项目上方的无实体 agent;它跑在一个具体的工作目录里,并深度感知该目录的结构。一组约定决定了 Claude 能看到什么、它如何为自己做配置。其中最重要的一个约定,是 `.claude` 这个目录。它可以放在一个项目的根目录下,也可以放在用户的 home 目录下,或者两边都放——配置就住在那里。Skill 住在 `.claude/skills/` 下的子目录里;subagent 的定义住在 `.claude/agents/` 下的文件里;hook 的定义住在 `.claude/settings.json`(或者对应的 user-level 配置文件)里。这个目录的结构,事实上就是"开发者那个 Claude Code 环境"的源代码。

There is a hierarchy at play. User-level configuration, which lives in the home directory, applies to every Claude Code session the user opens, on every project. Project-level configuration, which lives in the `.claude` directory at the root of a specific project, applies only when Claude Code is opened inside that project. When both are present, the project-level configuration generally takes precedence for project-specific concerns, while the user-level configuration provides a stable personal baseline. A developer who works on five different projects does not need to redefine her personal commit-message skill five times. She defines it once at the user level, and it travels with her. When a specific project

needs a project-specific skill, a pre-commit hook that runs that project's specific test suite, or a subagent trained on that codebase's peculiar conventions, those live in the project's `.claude` directory and are committed to source control alongside the code itself.

这里存在一个*层级*。User-level 配置(放在 home 目录里)作用于这个用户在*每一个项目*上打开的*每一次* Claude Code 会话。Project-level 配置(放在某个具体项目根目录下的 `.claude` 里)只在 Claude Code 是在该项目里被打开时才生效。两者都存在时,通常 project-level 在"项目特有"的事务上具有更高优先级,而 user-level 提供一个稳定的"个人基线"。一个同时在做五个项目的开发者,不需要把"个人 commit message skill"重新定义五遍——在 user 级别定义一次,它会跟着她走。当某个项目需要"项目特有的 skill"、需要"跑该项目特定测试套件的 pre-commit hook"、或需要"针对该代码库独特规约训练过的 subagent"时,这些就放在该项目的 `.claude` 目录里,并随源代码一起被提交进版本控制。

Where Configuration Lives · 配置住在哪里

This last point deserves emphasis. The project-level `.claude` directory is part of the codebase. It is as much part of the codebase as the source files, the package manifest, or the README. It should be committed, reviewed in pull requests, and evolved deliberately. When a developer joins a team, the team's `.claude` directory travels to her automatically when she clones the

这最后一点值得多花点笔墨强调:project-level 的 `.claude` 目录,是*代码库本身的一部分*。它和源代码、package manifest、README 一样,都属于"代码库"。它应当被提交进 git、在 PR 中被评审、有意识地演进。当一位新开发者加入团队,她一 clone 仓库,团队的 `.claude` 就自动跟着她来到她本机——

repository. She inherits the team's skills, the team's subagents, and the team's hooks at the moment she starts working. She does not have to be told, orally, what the team's conventions are, because the conventions are encoded as configuration that Claude enforces for her. This is how a well-engineered environment spreads. Not through wikis. Not through onboarding videos. Through version-controlled configuration that the tool itself honors on the first session.

她一 clone 下来,团队的 skills、subagents、hooks 就同时落到了她手上。她不需要别人口头告诉她团队的规约是什么——那些规约,已经以"由 Claude 来替她执行"的配置形式编码好了。这就是"被工程化好的环境"如何扩散:不是通过 wiki,不是通过 onboarding 视频,而是通过"被版本管理、且被工具自身在第一个 session 就尊重"的配置。

> The fourth thing to understand is `CLAUDE.md`. This file is a special convention that Claude Code looks for automatically, and whose contents are added to the system context at the start of every session. `CLAUDE.md` is the place for information that applies to every interaction within a project. It is where one writes, this codebase uses TypeScript with strict mode; tests are in Vitest; authentication changes require careful review. `CLAUDE.md` is not a skill, and it is not a substitute for one. A skill is targeted; it activates when its description matches the user's intent. `CLAUDE.md` is ambient; its contents are always loaded. The right things to put in `CLAUDE.md` are things that need to be true at all times. The wrong things to put there are things that only matter when a specific kind of work is being done, because those things should be skills instead, and should be activated only when they are needed.

第四件要理解的事,是 `CLAUDE.md`。这个文件是 Claude Code 自动寻找的特定约定:它的内容,会在每个 session 启动时,被加进 system context 里。`CLAUDE.md` 是写"对项目内每一次交互都生效"信息的地方。比如:"本代码库使用 TypeScript 的 strict 模式;测试用 Vitest;鉴权改动需要仔细 review"。`CLAUDE.md` 不是一个 skill,也不是 skill 的替代品。Skill 是*靶向*的——只有当它的 description 命中用户意图时,它才会激活;`CLAUDE.md` 是*环境性*的——它总是被加载。适合放进 `CLAUDE.md` 的,是"任何时候都需要为真"的事;不适合放进去的,是"只有在做某一类工作时才有意义"的事——后者应该写成 skill,只在被需要时才激活。

> This distinction, ambient versus targeted, is one of the design questions the reader will be making over and over again throughout the book. When a piece of knowledge belongs in `CLAUDE.md`, and when it belongs in a skill, is a judgment call. A rule of thumb that will serve well: if the knowledge is relevant to almost every action in the project, put it in `CLAUDE.md`. If the knowledge is relevant only to a specific kind of action, put it in a skill, and write the skill's description so that the skill fires when that kind of action is being requested. This rule will be refined as the book proceeds, but it is a good starting place.

这条"环境性 vs 靶向"的区分,是读者在整本书里会反复做出的一个设计判断。"这条知识该放 `CLAUDE.md`,还是该放 skill 里",是一个判断题。一条很好用的经验法则:如果这条知识对项目里几乎所有动作都相关,放 `CLAUDE.md`;如果只对某一类动作相关,放 skill,并把 skill 的 description 写好,使得"该类动作被请求时"它能被触发。这条法则在本书后续会逐步精化,但作为出发点,够用了。

> The fifth thing to understand is the context window. Every model has a finite context window, and every session spends that window progressively as it runs. The context window contains the system prompt, the assembled bundle of skills and subagents and hooks that apply to this session, the `CLAUDE.md` content, the history of the conversation so far, and the outputs of every tool call Claude has made. The context window fills up. When it fills up, the quality of the model's responses degrades, because the model has to

第五件要理解的事,是*上下文窗口*(context window)。每个模型都有一个有限的上下文窗口;每次会话,在它运行的过程中,都会把那个窗口一点一点地花掉。上下文窗口里装着的,包括:系统提示、为本次 session 装配好的 skills / subagents / hooks、`CLAUDE.md` 的内容、到目前为止的对话历史,以及 Claude 调用过的每一个工具的输出。上下文窗口会满。满的时候,模型回答的质量就会下降,因为模型要在更多噪声里费力地去定位真正相关的信息。

> work harder to find the relevant information among the noise. Eventually, the context window fills up entirely, and something has to give.

到最后,模型不得不在大量噪声里苦搜,等窗口彻底塞满,总得有东西得让位。

Context as a Scarce Resource · 把上下文视作稀缺资源

> This is why subagents matter so much. A subagent, when it runs, has its own fresh context window. The main session dispatches a task to the subagent, the subagent does the work in its own private context, and only the subagent's final output is added back to the main session's context. The dozens of tool calls the subagent may have made, the files it may have read, the drafts it may have discarded, none of it enters the main session. The main session stays clean. The developer who designs subagents well is, in effect, designing a context budget, a plan for what deserves to live in the main session's precious memory and what should be delegated to a subprocess that will return only its conclusion.

这就是 subagent 如此重要的原因。一个 subagent 跑起来时,自带着一个全新、干净的上下文窗口。主会话把任务派给 subagent;subagent 在自己的私有上下文里干完活;只有 subagent 的"最终输出"会被加回主会话的上下文。subagent 内部做过的那几十次工具调用、它读过的文件、它

丢掉过的草稿,统统不进主会话。主会话因此保持干净。一个把 subagent 设计得好的开发者,事实上是在设计一份"上下文预算"——一份计划:什么值得住进主会话宝贵的记忆里、什么应该外包给一个只回传结论的子进程。

> The sixth thing to understand is the lifecycle. A Claude Code session is not a single event. It has stages. There is the session start, when the environment is assembled and the initial context is loaded. There is the prompt submission phase, when the user's input is appended to the context. There is the tool use phase, when Claude decides to call a tool; this phase is composed of an approval step, the actual tool call, and the observation of its result. There is the stop phase, when Claude decides the task is done. There is the session end, when the process terminates. Each of these stages is a moment at which something can be made to happen. The hook system is precisely the mechanism by which a developer attaches behavior to these moments. When a developer wants something to happen before every tool use, she attaches a PreToolUse hook. When she wants something to happen after Claude finishes speaking, she attaches a Stop hook. The lifecycle is the grid of available moments, and hooks are the pins that attach behavior to the grid.

第六件要理解的事,是*生命周期*。一次 Claude Code 会话并不是一个"单一事件";它有阶段。SessionStart——环境被装配、初始上下文被加载。Prompt 提交阶段——用户的输入被追加进上下文。Tool 调用阶段——Claude 决定调用一个工具;这一阶段由三步组成:批准、实际调用、观察结果。Stop 阶段——Claude 判定任务完成。SessionEnd——进程终止。每一个阶段,都是"可以发生点什么"的时刻。Hook 系统,正是开发者用来把行为挂到这些时刻上去的机制。想在每次工具调用之前发生点什么,就挂一个 PreToolUse hook;想在 Claude 说完话后发生点什么,就挂一个 Stop hook。生命周期是"可挂载时刻"组成的一张网格,hook 是把行为挂到网格上的那根钉子。

> A seventh observation, and the last of the anatomy, concerns the way Claude Code thinks about safety. Every tool use is, by default, a point of trust. The language model decides, based on the user's prompt and the context bundle, that a certain tool should be called with certain arguments. If that decision is wrong, and if the tool has consequences in the outside world, the wrong decision becomes a real event. Claude Code mitigates this in several ways that the reader should be aware of. Tool permissions can be

第七条,也是解剖学中的最后一条,关乎 Claude Code 怎么思考*安全*这件事。默认情况下,每一次工具调用都是一个"信任点":语言模型根据用户的 prompt 和上下文包袱,决定"某个工具应当以某组参数被调用"。如果这个决定是错的——而那个工具又在外部世界有真实副作用——那么这个错误

的决定,就变成了一件真实发生的事。Claude Code 用几种方式来缓解这个风险,读者应该知道:工具权限可以

> restricted at the session, project, or user level, so that certain tools are never available in certain contexts. Approvals can be required for specific categories of tool use, so that the developer retains a human check on high-impact operations. And hooks, as will be seen in the fourth part of this book, can intercept tool uses before they happen and reject them on any grounds the developer can encode in a shell command. Safety, in this model, is not a property of the language model alone. It is a property of the configured environment, and the developer is the one who configures it. This responsibility is not heavy; it mostly consists of making good defaults and then forgetting about them. But it is real, and the reader who takes the time to set good defaults early will never have to think about them later, which is the best outcome a safety system can offer.

在 session、project、user 三个层级上被限定,从而某些工具在某些场景下压根不会出现;特定类别的工具调用可以被要求人工批准,从而在"高影响操作"上保留一道人为闸门;而 hook(本书第四部分会专门讲)则可以在工具调用发生前拦截它,按开发者能写成 shell 命令的任意理由拒绝它。在这个模型下,安全*不是*语言模型自身的属性——它是"被配置好的环境"的属性,而配置环境这件事,由开发者本人来做。这份责任并不重:大部分情况下,就是把好的默认设好,然后忘了它。但它是真的;愿意在早期就把好的默认设好的读者,日后就不必再为这件事操心——而这正是一个安全系统能给出的最好结果。

> This is the whole machine, seen from enough height to be understood in a single glance. A language model, wrapped in an agent loop, wrapped in a file-system-aware configuration system, wrapped in a lifecycle of named events, wrapped in a session that the user drives by typing prompts. Every sophisticated use of Claude Code that the reader will encounter in this book, and every sophisticated use she will design on her own in the years to come, is some configuration of this machine. There is no hidden trick. There is no magic. There is a well-defined system, and the question is only how well the developer understands that system and how deliberately she shapes it.

这就是这台机器的全貌——从一个足够高、足够一眼看懂的角度看过去:一个语言模型,被一条 agent 循环包着;这条 agent 循环,又被一个"懂文件系统"的配置系统包着;这个配置系统,又被一

个"由命名事件组成的生命周期"包着;这个生命周期,又包在一个"由用户敲 prompt 来驱动的会话"里。读者在本书里会遇到的每一个 Claude Code 高级用法——以及你未来自己设计出来的每一个——都是这台机器的某种"配置形态"。没有暗藏的诀窍。没有魔法。是一台定义良好的系统,问题只在于:你多懂它,你多有意地去塑造它。

Composition by Default · 组合是默认动作

> There is also a subtler property of Claude Code worth surfacing while the anatomy is fresh: the environment is not static. Anthropic has been evolving the tool continuously, adding new lifecycle events, refining the skill system, expanding what subagents can do, broadening the matchers available to hooks. The developer who engineers her environment in this tool is aiming at a moving target, and that is not a drawback. It is a feature. The principles that the book is about to teach are stable principles, rooted in the shape of the loop and the lifecycle rather than in any specific version of the tool. A developer who understands skills, subagents, and hooks in the sense this book defines them will be able to absorb new features as they ship, because she will recognize each new feature as a variation on a theme she already knows. The framework is, in that respect, a defense against the churn of the underlying software. The developer who has an opinion about how her

趁"解剖图"还新鲜,Claude Code 还有一条更隐微、但同样值得点出的特性:这个环境*不是*静态的。Anthropic 一直在持续演化这个工具——加入新的生命周期事件、改进 skill 系统、扩展 subagent 能做的事、放宽 hook 的匹配规则。在这款工具里工程化自己的环境的开发者,瞄的是一个移动靶——而那不是缺点,是特性。本书要教的原则,都是稳定的原则,它们根植于"那条循环 + 那个生命周期"的形状,而非某个特定版本的工具。真正按本书定义去理解 skills、subagents、hooks 的开发者,会很容易吸收新功能——因为她会认出"每一个新功能,不过是一个她已经知道的主题的变奏"。从这点看,这套框架其实是对"底层软件一直在变"的一种防御。一个对自己的

> environment should be shaped is the developer who can evaluate new features on their merits, keep what serves her work, and ignore what does not. The developer who has no such opinion will be pushed around by whatever the current release notes emphasize.

环境"应该长成什么样"心里有主张的开发者,才能按价值评估新功能,留下对自己工作有用的,忽略不相关的;而没有这种主张的开发者,只能被每一版 release notes 强调的热点推着走。

> A short diagnostic exercise is worth performing here, because it will make the abstractions of this chapter much more concrete. The reader is encouraged, when she has a moment at her own terminal, to open a project in Claude Code and explicitly map its anatomy. What is in the project's `CLAUDE.md` file, if one exists? What skills live in the project's `.claude/skills` directory, and what do their descriptions say? What subagents are defined, and what do their system prompts establish about their specialization? What hooks are configured in the project's settings file, and at which lifecycle events do they fire? This exercise, which should take only fifteen or twenty minutes, often reveals two things. First, it reveals that the environment the developer has been operating in has more shape than she realized, because previous developers or the team standard has already put some configuration in place. Second, it reveals the gaps, which is to say, the places where no configuration exists and where the developer has been filling the gap with her own attention, session after session. Those gaps are the raw material for the rest of the book. Every gap is a candidate for a skill, a subagent, or a hook, and by the end of the book the reader will have a considered view on which is appropriate for which gap.

这里值得做一次"小诊断练习"——它会令本章的抽象概念具体得多。读者在自己终端方便的时候,被鼓励去打开一个 Claude Code 项目,显式地"画出"它的解剖图:项目的 `CLAUDE.md` 里写了什么(如果存在的话)?项目的 `.claude/skills` 目录里有哪些 skill,它们的 description 在说些什么?定义了哪些 subagent,它们的 system prompt 是怎样确立自己的"专门化"的?项目的 settings 文件里配置了哪些 hook,它们在哪些生命周期事件上触发?这个练习花不了多久——十五到二十分钟;但常常会揭示两件事。第一,她一直在里面操作的这个环境,实际上比她以为的"更有形状"——之前的开发者或者团队规范已经在那里放了一些配置。第二,它会揭示"缺口"——也就是那些"没有配置、她一直在用自己的注意力一次次去补"的地方。这些缺口,就是本书后半部分的原材料。每一个缺口,都是 skill、subagent、hook 的潜在候选;而读到最后,你会形成一个"有理由的看法"——哪种缺口适合用哪种原语去补。

> One final note is worth making before this chapter closes. The word configuration has been used many times in these pages, and it is a word that can sound dry and administrative. It is not. In the world of modern software, configuration is power. The most influential tools in any developer's life are tools whose default behavior can be bent by configuration to fit that developer's specific needs. A text editor is an example; the raw editor is impressive, but an editor configured over years with the right extensions, keybindings, and snippets becomes something much more powerful than the sum of its defaults. A shell is another; the raw shell is capable, but a shell configured with the right aliases, functions, and prompt

becomes an extension of the developer's intentions rather than a tool she is constantly wrestling with. Claude Code is, in this respect, more like a modern editor or a well-configured shell than like a chat interface. It is meant to be configured. It is designed with configurability as a first-class

本章收尾前,还值得点一笔:"配置"这个词在前面出现了很多次,听上去可能有点干、有行政味。但它不是。在现代软件的世界里,*配置就是权力*。任何一个开发者生命里最有影响力的工具,都是"其默认行为可以被配置、从而贴合该开发者具体需求"的工具。文本编辑器就是一个例子:原始的编辑器已经很强;但一个用若干年精心配置出合适的扩展、快捷键、代码片段的编辑器,会比它的所有默认值的简单相加更强。Shell 也是一个例子:原始的 shell 也能干很多事;但一个用合适的 alias、function、prompt 配置出来的 shell,会变成"开发者意志的延伸",而不是她要持续与之搏斗的工具。从这点看,Claude Code 比起"聊天界面",更接近一款现代编辑器或一款被精心配置过的 shell。它生来就*应当*被配置。它把"可配置性"作为头等公民

concern. The developer who treats Claude Code as a tool to be configured, rather than merely used, is the developer who will extract the full value from it. Everything in this book is an attempt to teach that configuration seriously and well.

来设计。开发者若把 Claude Code 当成"待配置的工具"而不是"被使用的工具",才是能从中榨出全部价值的那种开发者。本书所有的内容,都在试图把"配置"这件事教得严肃、且教得好。

With the anatomy clear, the chapter that follows introduces the three-pillar framework that organizes all of that configuration into a coherent system of thought. The framework is the single most important concept in this book, and the rest of the book is, in a sense, a careful elaboration of it. The reader who has internalized the anatomy described here is now ready to see the framework for what it is.

"解剖图"已经清楚之后,下一章将引入那套"三支柱"框架——它能把所有这些配置,组织成一套连贯的思维系统。这个框架,是本书里最最重要的一个概念;从某种意义上说,后面整本书,都是对它的一次精心展开。已经把前面这套解剖图内化了的读者,现在可以看到这个框架的"真身"了。

CHAPTER THREE · 第三章

Skills, Subagents, and Hooks: The Three-Pillar Framework

技能、子代理与钩子:三支柱框架

> *"Architecture is the art of how to waste space beautifully, or how to save it meaningfully. It is never neutral."*
>
> — Philip Johnson, adapted

"建筑,是一门如何把空间漂亮地浪费掉、或把它有意义地节省下来的艺术。它从来不是中立的。"

—— 菲利普·约翰逊(改写)

Every discipline eventually discovers that its power comes less from any single technique and more from the framework that organizes its techniques into a coherent whole. In medicine, the framework of anatomy organizes the study of the body. In music, the framework of harmony organizes the study of chords. In software engineering, frameworks of architecture organize patterns that would otherwise feel like a disordered bag of tricks. This chapter introduces the framework that organizes everything in this book. It is a simple framework with only three parts, but the simplicity is deceptive. Once the reader has internalized these three parts and the relationships between them, every subsequent chapter in the book reads as a natural elaboration. Before the reader has internalized them, the chapters look like a catalog of features. After, they look like a single unified argument.

任何一门学科最终都会发现:它的力量,与其说来自任何一项单独的技术,不如说来自"把那若干技术组织成一个连贯整体"的框架。在医学里,解剖学的框架组织了"对身体的认识";在音乐里,和声的框架组织了"对和弦的认识";在软件工程里,各种架构框架把那些"否则会显得像一袋零散技巧"的设计

模式组织起来。本章要介绍的那个框架,就是组织本书所有内容的那个。它很简单,只有三块;但这种简单有迷惑性。一旦你把这三块以及它们之间的关系内化,后面每一章读起来都像"自然而然的展开";在内化之前,这些章节看起来像一份特性目录;内化之后,它们看起来则像一段统一的论证。

> The framework is this. Claude Code mastery is built on three pillars. The first pillar is Skills, and it supplies capability. The second pillar is Subagents, and they supply parallelism and specialization. The third pillar is Hooks, and they supply determinism. Every sophisticated workflow is some combination of these three, and the real art of mastery lies not in any one of them in isolation but in how they are composed.

框架是这样的:Claude Code 的"掌握",建立在三根支柱之上。第一根是**Skills(技能)**,它提供*能力*。第二根是**Subagents(子代理)**,它们提供*并行*和*专门化*。第三根是**Hooks(钩子)**,它们提供*确定性*。每一个真正"成熟"的工作流,都是这三者的某种组合;而"掌握"的真正艺术,不在其中任何一根独立的存在里,而在它们是如何被组合在一起的。

> Begin with the first pillar. A Skill is the answer to a specific question: how does a developer teach Claude to do something consistently, across sessions, across projects, without repeating herself? The question looks simple, but it is the root of an enormous amount of developer frustration. A developer writes a prompt that produces excellent output. She is pleased. She moves on. Three weeks later, she wants that same kind of output again, and she cannot remember exactly how she phrased the prompt. She tries to

从第一根开始。Skill 回答的是一个具体的问题:开发者要怎样,才能跨 session、跨项目地教 Claude 始终如一地做某件事,而不必一遍遍重复自己?问题看上去简单,却是大量开发者挫败感的根源。开发者写了一段 prompt,得到了很棒的输出,很高兴,继续做别的事。三个星期后,她想再要一次同类输出,却已经想不起当时的 prompt 究竟是怎么措辞的。她试着去

> reconstruct it. The result is not quite as good as before. She keeps trying. Eventually, the reconstruction converges to the original, but she has wasted thirty minutes. This is a daily experience for every Claude Code user who has not adopted skills. Skills are the cure.

还原它。还原出来的,没原来那么好。她继续试,最后勉强拼回原样,但半小时已经搭进去了。这是每一位"还没有采用 skill"的 Claude Code 用户的日常。Skill 就是解药。

> A skill is, in its essence, a promise. It is a promise that whenever a certain kind of task is requested, Claude will remember how to do that task well, because the instructions for doing it well have been captured in a file that Claude will read. The file is called `SKILL.md`. It lives in a folder named after the skill, inside either the user's personal skill library or the project's skill library. It has a structured head and a free-form body. The head declares the skill's name, its description, and the tools it is allowed to use. The body contains the instructions themselves, in plain prose, with examples, references, and any other material that would help a capable reader do the task well for the first time.

一个 skill,在本质上,是*一份承诺*。承诺的是:只要某一类任务被请求,Claude 就会记得"把这件任务做好"——因为"如何做好它"的指令,已经被捕获到一个 Claude 会去读的文件里了。那个文件叫 `SKILL.md`。它住在一个以 skill 命名的目录里,而该目录可以放在用户的个人 skill 库里,也可以放在项目的 skill 库里。它有一个"结构化头部"+ 一个"自由形式正文"。头部声明 skill 的名字、description,以及它被允许使用的工具;正文是真正的指令,以普通散文形式呈现,加上示例、参考,以及任何能帮一位有能力的读者"第一次就把这件任务做对"的材料。

> When Claude Code starts a session, it discovers all of the available skills. It does not load their full contents at once, because that would waste the context window. It loads only the descriptions, because the descriptions are enough to tell Claude when each skill is relevant. When a user issues a prompt, Claude scans the descriptions of the available skills and recognizes which ones, if any, apply to the prompt. If a skill applies, Claude reads its body and uses those instructions to shape its response. This is what progressive disclosure means in the context of skills: only what is relevant gets read, at the moment it becomes relevant, and nothing more.

Claude Code 启动一次 session 时,它会"发现"所有可用的 skill。但它不会一次性把它们的全部正文都加载进来——那样会浪费上下文窗口。它只加载 description;description 已经够用,足以让 Claude 判断"每个 skill 在什么时候相关"。当用户敲下一段 prompt,Claude 会扫一遍可用 skill 的 description,识别出"哪些与本段 prompt 相关"。如果某个 skill 命中,Claude 就去读它的正文,并用那些指令来塑造自己的回答。这就是 skill 语境下的*渐进式披露*(progressive disclosure):只有相关的东西才会被读,只在"它变得相关的那一刻"被读,再无其他。

> This mechanism is what makes skills practical. A developer can accumulate dozens or even hundreds of skills without paying a context-window tax for the ones she is not using. Only the skills that match the current request cost tokens. Every other skill sits quietly in the library, waiting for the kind of request that would activate it. A skill, in this sense, is a dormant capability, and a skill library is a collection of dormant capabilities. When the library has been designed well, the developer's work activates the right capabilities at the right moments, and a substantial portion of the developer's daily knowledge work runs itself.

正是这套机制,让 skill 变得实用。开发者可以积累几十、甚至几百个 skill,却不必为那些"没在用"的 skill 付上下文窗口的税。只有与当前请求匹配的 skill 才消耗 token;其他每一个 skill,都安静地待在库中,等待那种会激活它的请求出现。从这个意义上说,skill 是一种*休眠中的能力*;而 skill 库,就是一组"休眠中的能力"的集合。当这个库被设计得很好,开发者的工作就会在恰当的时刻激活恰当的能力;而她日常知识工作中的相当一部分,会自己跑起来。

Skills in Practice · Skill 在实战中

> What does a skill look like in practice? Consider a simple example. A developer writes a lot of pull request descriptions, and has developed a specific format she likes: a one-sentence summary, a bulleted list of what changed, a note about testing, and a note about risk. She has typed this format into prompts hundreds of times. She sits down one evening and writes a skill called `pr-description`. In `SKILL.md`, she writes a description that says, activate this skill whenever the user asks for a pull request description, PR description, or PR body. In the body, she lays out the format in detail, with an example filled in. She saves the file, restarts her Claude Code session, and the next time she types, give me a PR description for this branch, the skill fires. The description that comes back follows her format exactly. She does not explain the format in the prompt. She does not need to. The skill remembers for her.

Skill 在实战中长什么样?看一个简单的例子。一位开发者要写大量 PR 描述,而且她已经形成了一种自己偏好的固定格式:一句摘要、一个"改了什么"的项目列表、一段关于测试的说明、一段关于风险的说明。她已经在 prompt 里手敲过这个格式几百次了。某个晚上她坐下来,写了一个叫 `pr-description` 的 skill。在 `SKILL.md` 里,她写了一段 description:任何时候,只要用户要 pull

request description、PR description 或 PR body,就激活这个 skill。在正文里,她把这个格式详细展开,还附上一个填好的示例。她保存文件,重启 Claude Code session,下次她再敲"给我这个分支写一段 PR 描述"——skill 立刻触发;Claude 给回的描述,严格遵守了她的格式。她不需要在 prompt 里再解释一次。她不需要。skill 替她记着。

> This is a small example, but it is representative. Every skill is, at its core, the codification of something that used to live only in the developer's head and her prompting habits. Skills make that knowledge durable. They are how a developer turns her accumulated expertise into configuration, and through configuration, into compounding leverage.

这是一个很小的例子,但有充分的代表性。每一个 skill,在最底层,都是"把原本只活在开发者脑子里、和她的 prompt 习惯里的那点东西"做一次"制度化"。Skill 让那份知识变得耐久。它是开发者把"自己积累下来的专业"转化为"配置"、再借由配置转化为"复利式杠杆"的途径。

> Now turn to the second pillar. A Subagent is the answer to a different kind of question: how does a developer do many kinds of work at once, without drowning her main session in the noise of all those kinds of work, and without losing the specialized focus that each kind of work deserves?

现在转向第二根支柱。Subagent 回答的是另一类问题:开发者要怎样,才能在同时做很多类工作时,既不让她主会话被这些工作的噪声淹没,又不丢失"每一类工作本该享有的那种专门化聚焦"?

> Every non-trivial development task is really several tasks in a trench coat. A feature request involves reading existing code, designing new code, writing the new code, writing tests for it, updating documentation, and reviewing the whole for quality. A single Claude session can do all of these things in sequence, and it often does, but there is a cost. Every file Claude reads in the course of understanding the existing code takes up space in the context window. Every false start, every draft, every experiment pollutes the history of the session. By the time Claude is writing the tests, the context window is crowded with exploration residue that has nothing to do with testing. The tests it writes, as a result, are sometimes shaped by irrelevant context that it cannot easily ignore.

每一个不那么琐碎的开发任务,本质上都是"穿了一件风衣的若干个任务"。一个 feature 请求,实际包含:读现有代码、设计新代码、写新代码、为之写测试、更新文档、再对整体做一轮质量 review。一个 Claude session 当然可以按顺序把这一串事都做了——它经常也确实这么做——但那是有代价的。Claude 为了弄懂现有代码而读的每一个文件,都占着上下文窗口的位;每一次走偏

的尝试、每一版被丢掉的草稿、每一次试错,都把会话的历史污染了一层。等 Claude 终于写到测试时,上下文窗口里已经堆满了"和测试毫不相关的探索残余"。它最后写出来的测试,因而不时会被那些"与测试无关、却无法轻易忽视"的上下文所塑造。

> A subagent solves this by giving a specific kind of work its own isolated environment. When the main session needs tests written, it dispatches a test-writer subagent. The subagent has its own system prompt, tuned for the craft of writing tests. It has its own set of allowed tools, which

Subagent 的解法,就是把"某一类工作"放进一个*隔离*的环境里。主会话需要"写测试"时,就派出一个 test-writer subagent。Subagent 带着自己专为"写测试这门手艺"调教过的 system prompt;带着自己被允许使用的工具集,那个集合——

> probably includes reading code and writing new test files but probably excludes making commits or touching production configuration. It has its own context window, which starts empty and fills only with the information needed for the specific test-writing task at hand. When the subagent finishes, it returns its output to the main session. The main session receives the tests. The entire exploration the subagent did to produce those tests does not enter the main session's context. The main session stays focused.

很可能包含"读代码、写新测试文件",但很可能不包含"做 commit、动生产配置"。它还有自己的上下文窗口,启动时是空的,只会填进"为眼下这件具体的写测试任务"所需要的信息。Subagent 干完活,把它的输出回传给主会话;主会话收到那批测试。Subagent 为了产出这批测试所进行的那一整段"探索",一个字都不会进主会话的上下文。主会话因此保持聚焦。

> This architecture has several virtues that are worth naming. First, it saves context, which directly improves the quality of everything else the main session does. Second, it allows specialization, because each subagent can be tuned for its specific kind of work, with a role definition and tool restrictions appropriate to that work. Third, it allows parallelism, because multiple subagents can run at the same time, each on an independent piece of a larger task, and the main session can synthesize their outputs. A developer with a well-designed subagent library is able to dispatch a security review, a performance

> audit, a documentation pass, and a test-coverage check simultaneously on a single pull request, and receive four independent reports back. A single-threaded session could never do this well.

这套架构,有几个值得点名的优点。第一,它节省上下文——这直接提升了主会话做的所有其他事的质量。第二,它允许"专门化"——因为每个 subagent 都可以为自己的那类工作调教一遍,带上适合该工作的角色定义和工具限制。第三,它允许"并行"——多个 subagent 可以同时跑,各自分头处理一个更大任务的独立子块,主会话最后再把它们的输出综合起来。一份 subagent 库设计得好的开发者,能够在同一份 PR 上同时派一个安全审查、一个性能审计、一轮文档过审、一次测试覆盖率检查,并拿到四份独立报告;单线程的 session 永远做不好这件事。

Subagents in Practice · Subagent 在实战中

> Subagents, like skills, are defined in the `.claude` directory. Each subagent is a file that declares its name, its description, its allowed tools, and its system prompt, the last of which is where the specialization actually lives. The system prompt of a code-reviewer subagent is different from the system prompt of a test-writer subagent, because code review and test writing are genuinely different crafts. The art of subagent design, which later chapters will treat in depth, is in part the art of writing these focused system prompts well.

Subagent,和 skill 一样,也在 `.claude` 目录里定义。每个 subagent 是一个文件,声明它的名字、description、可用工具,以及它的 system prompt——最后这一项,才是"专门化"真正栖身之处。一个 code-reviewer subagent 的 system prompt,与一个 test-writer subagent 的 system prompt,不会一样;因为 code review 和写测试,确实是两门不同的手艺。"subagent 设计"这门艺术(后面章节会深入讲),一部分就是把这些"聚焦的"system prompt 写得好的艺术。

> There is one further property of subagents that deserves mention before leaving the second pillar. Subagents can themselves invoke skills. A test-writer subagent, when it starts, can have access to a skill that encodes the project's test conventions, and it can activate that skill just as the main session would. This is where the framework begins to compose. A subagent is not merely a specialist; it is a specialist who inherits the knowledge the developer has encoded into the environment. The specialist-plus-skill

在离开第二根支柱之前,subagent 还有一条值得提一提的性质:subagent 自身也可以调用 skill。一个 test-writer subagent,在它启动时,可以拿到一个"把项目测试规约编码在内"的 skill,并且可以像主会话那样去激活它。这是"框架"开始组合起来的地方。Subagent 不仅仅是"一个专家";它是一个*继承了开发者已经编码进环境里的知识*的专家。"专家+skill"

> combination is strictly more powerful than either alone, and the real art of building a Claude Code platform involves designing these combinations deliberately.

的组合,严格来说,比任何单用其一都要更强;搭建一套 Claude Code 平台的真功夫,正在于把这些组合*有意识地*设计出来。

The Third Pillar · 第三根支柱

> A Hook is the answer to the third question: how does a developer ensure that the things that must happen always happen, not merely most of the time?

Hook 回答的是第三个问题:开发者要怎样,才能确保那些"必须发生的事"*总是*发生——而不只是"大多数时候"发生?

> Language models are probabilistic. This is a statement of design, not of failure. A probabilistic system produces different outputs for the same inputs, and this is what allows a language model to handle the open-ended variety of human requests. The trade-off is that a probabilistic system cannot be trusted to always do anything. It can be coaxed, in most cases, into doing a thing reliably, but reliably is not the same as always. For the things that must happen always, probabilistic behavior is not enough.

语言模型是概率性的(probabilistic)。这是一个设计层面的陈述,而不是一句失败判决。一个概率系统,会对"相同的输入"产出不同的输出——这恰恰是语言模型能够处理人类请求那种"开放式多样性"的原因。代价是:一个概率系统,不可能被绝对地信任去做任何一件事。它可以在大多数情形下被哄得"可靠地做某事",但"可靠"和"总是"并不是一回事。对于那些"必须*总是*发生"的事,概率性行为远远不够。

> Consider the category of things that must happen always. Tests must be run before commits. Linters must be run after edits. Secrets must never be written to disk. Production configuration files must never

> be edited without explicit confirmation. Long sessions must not terminate silently without a summary being logged somewhere the developer can review. Each of these is a requirement where the acceptable failure rate is zero. A ninety-eight percent success rate is not enough. A developer who tells Claude, by prompt alone, to always run tests before committing, will find that in ninety-eight out of a hundred cases Claude does exactly that. In the other two, something else happens. Usually nothing bad. Occasionally something very bad. The whole point of the rule was to prevent the very bad thing, and a rule that fails two percent of the time is barely a rule at all.

请看"必须总是发生"这类事的清单:提交之前必须跑测试;编辑之后必须跑 linter;机密绝不允许落盘;生产配置文件未经明确确认绝不允许编辑;长 session 结束绝不允许悄无声息——必须把一段摘要记到开发者能查看的地方。每一项,都是"可接受失败率 = 0"的需求。98% 的成功率根本不够。某位开发者只用 prompt 告诉 Claude"提交前必须跑测试",他会在一百次里看到有九十八次 Claude 照做了;剩下两次,会发生别的事。通常没什么大碍;偶尔,会非常糟糕。整条规则的存在意义,本来就是为了防范那件"非常糟糕的事";而一条 2% 的时候会失效的规则,几乎根本算不上规则。

> A hook is the mechanism by which a developer can, at specific moments in the Claude Code lifecycle, insert a deterministic shell command that runs whether Claude remembers or not. If the hook fails, the lifecycle is interrupted. If the hook succeeds, the lifecycle continues. The language model does not get to override the hook, because the hook is not part of the language model's loop. The hook is outside the loop. The hook is wrapped around the loop.

Hook,就是一套机制:开发者可以在 Claude Code 生命周期的特定时刻,插入一条确定性的 shell 命令,不管 Claude 记不记得,这条命令都会跑。Hook 失败,生命周期就中断;hook 成功,生命周期就继续。语言模型无从覆盖 hook,因为 hook 本就不在语言模型那个循环里。Hook 在循环*之外*。Hook *包在*循环的外面。

The Hook Layer · 钩子层

← Chapter 3 上一页

后一页 →

This placement, outside the loop and around it, is the whole source of the hook's value. Hooks do not replace the language model's judgment; they constrain the environment in which the language model is making its judgments. A hook that refuses to let any tool use touch the `.env` file does not make the model smarter. It makes it unnecessary for the model to be perfectly vigilant about the `.env` file, because the `.env` file is protected by a rule the model cannot break. This frees the model to think about the work that does require judgment, while the work that requires only enforcement is handled by the hook. The probabilistic engine is left to do the probabilistic work, and the deterministic requirements are handled deterministically, each in its proper place.

"放在循环之外、包在循环外面"这个位置,正是 hook 全部价值的来源。Hook 并不取代语言模型的判断;它约束的是语言模型*做判断所处的环境*。一个拒绝任何工具调用去碰 `.env` 文件的 hook,并没有让模型变得更聪明;它让模型不必再为 `.env` 文件保持那种"完美警觉"——因为 `.env` 文件被一条模型*无法打破*的规则保护着。这反过来,反而让模型可以把精力放回那些"确实需要判断"的工作上;而那些"只需要强制执行"的工作,交给 hook 去办。概率性引擎留在原位做它该做的概率性工作,确定性需求被确定性地处理——各司其职,各得其所。

Hooks are defined in the `settings.json` file, at either the user level or the project level. Each hook declares which lifecycle event it responds to, an optional matcher that narrows when it should fire, and the command to run. The command can be any shell command. It can be a one-liner or a call to a full script. It can read the context of the hook from standard input in a structured format, and it can signal approval or rejection of the current action through its exit code and output. The design is deliberately simple at the configuration layer, so that the complexity can live in the scripts themselves, where it belongs.

Hook 在 `settings.json` 文件里定义,既可以放在 user 级,也可以放在 project 级。每个 hook 声明:它响应哪个生命周期事件、一个可选的 matcher(用来收窄触发条件),以及要执行的命令。那条命令可以是任意 shell 命令——一行 one-liner,或者对一个完整脚本的调用;它可以从 stdin 按一种结构化格式读入 hook 的上下文,也可以通过 exit code 和输出来表态:批准、还是否决当前动作。这种设计在配置层刻意保持简单,目的是让复杂度留在脚本里——那才是它该待的地方。

Having named the three pillars, the natural next question is how they relate to each other. The short answer is that they are complementary, and they become far more powerful together than any of them could be alone. The longer answer is the composition principle, which is the central design principle of the book, and which deserves its own short treatment before this chapter closes.

三根支柱既然有了名,下一个自然的问题就是:它们彼此之间是什么关系?简短的回答是:它们是*互补*的,合起来用,远比任一单用要强大得多。稍长的回答是*组合原则*(composition principle)——它是本书最核心的设计原则,值得在本章收尾之前专门用一小段来谈。

> The composition principle states that any sophisticated Claude Code workflow will use all three pillars, because each pillar addresses a different class of problem, and real workflows contain all three classes of problem. Skills address the problem of how to do a task well. Subagents address the problem of how to keep different kinds of work separated. Hooks address the problem of how to enforce what must happen. A workflow that uses only skills is a workflow that can teach Claude to do good work but has no way to isolate different kinds of work and no way to guarantee enforcement. A workflow that uses only subagents has specialization but no stored knowledge and no enforcement. A workflow that uses only hooks

组合原则陈述的是:任何一套成熟的 Claude Code workflow,都会同时使用这三根支柱;因为每根支柱分别对应着不同类别的问题,而真实的 workflow 恰好同时包含这三类问题。**Skill(技能)**回答"如何把一件事做好"的问题;**Subagent(子代理)**回答"如何把不同类别的工作彼此隔离"的问题;**Hook(钩子)**回答"如何强制那些必须发生的事真的发生"的问题。一套只用 skill 的 workflow,能教会 Claude 把活干好,却没办法隔离不同类别的工作,也没办法保证执行;一套只用 subagent 的 workflow,有了"专门化",却没有沉淀下来的知识,也没有强制执行;一套只用 hook 的 workflow

← Chapter 3 上一页

后一页 →

> has enforcement but no capability and no specialization. Each pillar, alone, is a partial answer. All three, together, form a complete answer.

有强制执行,却没有能力,也没有专门化。每根支柱单用,都只是"半个答案";三根合在一起,才构成完整的答案。

> The composition can be illustrated with a simple image. Imagine a developer whose morning routine includes reviewing the previous day's pull requests. She has a skill called `pr-review` that encodes the specific kind of review she likes to write: substantive comments, clear suggestions, and a summary judgment. She has a subagent called `security-auditor`, specialized for flagging potential security issues, with access to the file system and to a small set of security-focused reference material. She has a hook on `PreToolUse` that blocks any attempt to approve a pull request that touches the authentication layer without her explicit confirmation.

可以用一幅简单的图景来演示这种"组合"。想象一位开发者,每天早晨的固定动作之一,是 review 前一天的 pull request。她有一个叫 `pr-review` 的 skill,把她喜欢写的那种 review 形式编码下来:实质性的评论、清晰的建议、一句总结性的判断。她有一个叫 `security-auditor` 的 subagent,专门用来标记潜在的安全问题,持有文件系统访问权,以及一小批聚焦于安全的参考材料。她有一条挂在 `PreToolUse` 上的 hook,任何试图批准"动到鉴权层"的 PR 的动作,都会被这条 hook 拦下,直到她本人显式确认。

> When she sits down in the morning and asks Claude Code to review a specific PR, the skill fires because the description matches. Claude reads the skill and structures its review accordingly. During the review, Claude dispatches the `security-auditor` subagent in parallel, so that security issues are surfaced by a specialist rather than by Claude's generalist attention. The subagent returns its findings. Claude synthesizes the review, including the security findings, according to the format the skill has established. If Claude, perhaps through a subsequent prompt, attempts to approve the PR and the PR touches the authentication layer, the hook fires, blocks the approval, and informs Claude that explicit human confirmation is required. At no point has the developer had to remember to do any of this. The environment remembered. The environment composed its three pillars on her behalf, quietly, exactly when each was needed.

早晨她坐下,让 Claude Code 去 review 一个具体的 PR。Skill 触发——因为 description 匹配上了;Claude 读进 skill,按它规定的结构来组织 review。在 review 过程中,Claude 把 `security-auditor` subagent 并行派出去,让安全问题由一位*专家*去发现,而不是由 Claude 那颗"什么都管一点"的注意力去顺手发现。Subagent 把发现回传;Claude 再把 review 整体(连同安全发现一起)按 skill 确立的格式综合出来。接下来,如果 Claude——比如由于用户又下了一条新 prompt——试图批准这个 PR,而这个 PR 又恰好动了鉴权层,hook 立刻触发,拦住批准动作,并告诉 Claude:"必须经人类显式确认"。从头到尾,这位开发者本人没有被迫去"记得"任何一步。是*环境*替她记着;是*环境*,在每一个需要它的时刻,悄然地把这三根支柱组合起来,替她把活干了。

Engineering in Miniature · 缩微之中的工程化

> This is what it looks like, in miniature, to engineer with Claude Code. Skills provide the knowledge. Subagents provide the specialized attention. Hooks provide the guarantees. The developer provides the architecture, once, and then spends the subsequent months and years harvesting the dividends of that architecture in every session she runs.

以缩微之姿,这正是"用 Claude Code 做工程化"应有的样子。Skill 提供知识;Subagent 提供专门的注意力;Hook 提供保证;而开发者本人,只需提供一次架构,然后在之后若干个月乃至若干年里,在她启动的每一个 session 中,反复收取这份架构的"股息"。

> The remaining five parts of this book are, at heart, a detailed elaboration of this framework. Part Two teaches skills in depth, from their anatomy to their composition, and sends the reader out of it with a working personal skill library. Part Three teaches subagents, from their fundamentals to the orchestration patterns that unlock parallelism, and sends the reader out of it

本书剩下的五个部分,本质上,都是这套框架的细密展开。第二部分深入讲 skill——从其解剖到其组合,读完后读者手上会有一个能跑的个人 skill 库;第三部分讲 subagent——从基础到那些能解锁并行能力的编排模式,读完后读者手上会有一支——

← Chapter 3 上一页

后一页 →

> with a working team of specialists. Part Four teaches hooks, from their event model to the design patterns that make them reliable, and sends the reader out of it with a hardened, secured, deterministic environment. Part Five is the composition layer, where the three pillars are assembled into a full personal platform, and two end-to-end case studies walk the reader through building that platform for specific kinds of work. Part Six closes the book by teaching how to ship, share, and evolve a Claude Code platform over time.

能跑的"专家团队";第四部分讲 hook——从事件模型到让它们可靠的设计模式,读完后读者手上会有一个加固、安全、确定性的环境;第五部分是组合层,把三根支柱拼成一个完整的个人平台,并通过两个端到端案例,带读者为某一类具体工作把那个平台真真切切地搭起来;第六部分作为收尾,讲如何把一套 Claude Code 平台发布出去、与人共享、并在时间中持续演化。

> The reader who has followed the argument of this chapter now has everything she needs to understand why the book is organized the way it is. The three pillars are the organizing structure. Every chapter is an elaboration of one of them, or of the composition between them. The journey is straightforward, and the destination is a Claude Code environment that the reader has genuinely engineered to fit the shape of her work.

跟到这里的读者,现在拥有了解"本书为何如此组织"所需要的一切。三根支柱就是整本书的骨架。每一章,要么是其中一根的展开,要么是彼此组合的展开。这段旅程是直白的;旅程的终点,是一个*真正按她工作的形状被工程化出来的* Claude Code 环境。

> A closing thought, offered in the spirit of calibrating expectations, is worth including here. The framework in this book is deliberately simple. Three pillars, one composition principle, a small number of recurring design questions. This simplicity is a strength, not a weakness. A framework that can be held entirely in working memory is a framework that a developer can actually use, in real time, in the middle of the day, while she is trying to decide whether a given piece of workflow friction should become a skill, a subagent, a hook, or nothing at all. A more elaborate framework, with a dozen categories and a dozen principles, would be impressive to read about and useless to apply. The three-pillar framework is sized for the working life of a human being who also has other things to do. That sizing is part of the design.

这里值得插入一段收束之语,用来校准预期。本书讲的框架,刻意保持简单:三根支柱,一条组合原则,再加上一小撮反复出现的设计问题。这种简单是一种长处,不是弱点。一个能完全装在工作记忆里的框架,才是开发者真能在白天、实时、当场用得起来的框架——彼时她要决定的是:这一处 workflow 摩擦,究竟该改造成一个 skill、一个 subagent、一个 hook,还是索性不动手。一套更"宏大"的框架,带十几条分类、十几条原则,读起来很过瘾,用起来却无从下手。"三根支柱"的体量,是为"一个还有其他事要做的普通人"的工作日量身定下的。这种"尺寸",本身也是设计的一部分。

> The simplicity also has an implication for the reader's relationship with the book. None of the chapters that follow require memorization. They require understanding, and the understanding is built on the

same three pillars, considered from different angles. A reader who has genuinely absorbed this chapter can, in principle, reason her way to many of the conclusions the later chapters will reach. The later chapters will spare her the effort, and will also introduce specific techniques, patterns, and configurations that would be hard to derive from first principles. But the

这份"简单",同样意味着读者与本书之间一种特定的关系。后面任何一章都不需要"背下来",需要的是"理解";而这种理解,仍然建在相同的三根支柱之上,只是换着不同的角度去审视。一个真的把这一章吸收了的读者,原则上,完全可以靠推演走到后面许多章要给出的结论。后面那些章节只是替读者省去推演的力气;同时,也会介绍一些光靠"从第一性原理推"很难推出来的具体技法、模式与配置。但那

← Chapter 3 上一页

后一页 →

backbone of the thinking is already in place. What remains is the detailed craft of applying it.

个"思考的骨架"其实已经搭好。剩下的,只是把这份骨架落到实处的细密手艺。

Part One closes here. Part Two begins with the deepest single topic in the book, which is the anatomy of the `SKILL.md` file, and it is there that the real work begins.

第一部分,到这里收束。第二部分从本书最深的单一主题起步——`SKILL.md` 文件的解剖;真正的活,从那里开始。

← Chapter 3 上一页

PART II: Skills(Ch 4) →